

Project Proposal

AION

Autonomous Incident Observation & Navigation

Project work for

B.E. in Computer Engineering

At

Department of Computer Engineering

Ajeenkya D Y Patil School of Engineering, Charholi Bk

Students

330114 - Aditya Bheke

330115 - Harshal Bhogawade

330116 - Malhar Bhoi

Guide

Dr. Yogesh Bahendwar

AY 2026-27

1. **Project Group ID:** 42
2. **Title of the Project:** AION – Autonomous Incident Observation and Navigation
3. **Domain:** Artificial Intelligence (AIOps), Large Language Models, DevOps (CI/CD), Full Stack Development
4. **Team Members:**

PRN	Roll No	Name	Role / Task Assigned	Email	Mobile	Sign
72331906F	330114	Aditya Bheke		aditya.bheke@dypic.in	7620166066	
72331908B	330115	Harshal Bhogawade		harshal.bhogawade@dypic.in	9373685636	
72331910D	330116	Malhar Bhoi		malhar.bhoi@dypic.in	8237832141	

5. **Sponsorship details, if any (Name, External Guide name and Designation with Signature, e-Mail ID):** Not Applicable
6. **Internal Guide Name (with signature of approval):** Prof. Anamika Jain
7. **Type of Project:** Software / Application-Based Project
AI-Powered Incident Detection, Root-Cause Analysis and Remediation Platform

Abstract

AION (Autonomous Incident Observation & Navigation) is an AI-assisted incident response and remediation platform that reduces the time, effort and risk involved in resolving software production incidents. In conventional incident management, engineers manually inspect large volumes of logs, trace the failure to a recent code change, write and test a fix, and run the deployment pipeline, using several disconnected tools.

AION connects these activities in a single, auditable **closed-loop workflow**. It collects structured application logs, deduplicates repeated errors into signatures, and detects incidents as error spikes against a baseline. Each incident is correlated with **Git commit and deployment history** using git blame, deployment timing and code-dependency signals to produce an explainable ranking of suspect commits.

A **Large Language Model (LLM)** layer, using the Claude API or a locally hosted model, receives a curated evidence pack and produces an evidence-cited root-cause analysis that is checked for grounding. AION then generates a **candidate code patch** with a regression test on an isolated Git branch, and validates it through syntax checks, a fail-to-pass test, the existing unit tests, a staging deployment and a replay of the failing production requests.

AI may investigate, analyse, propose and validate, but **only a human engineer can approve production deployment**, and the approval is bound to the exact commit that passed validation. Every step is recorded in an append-only audit trail, so incidents are resolved faster while deployment stays safe and accountable.

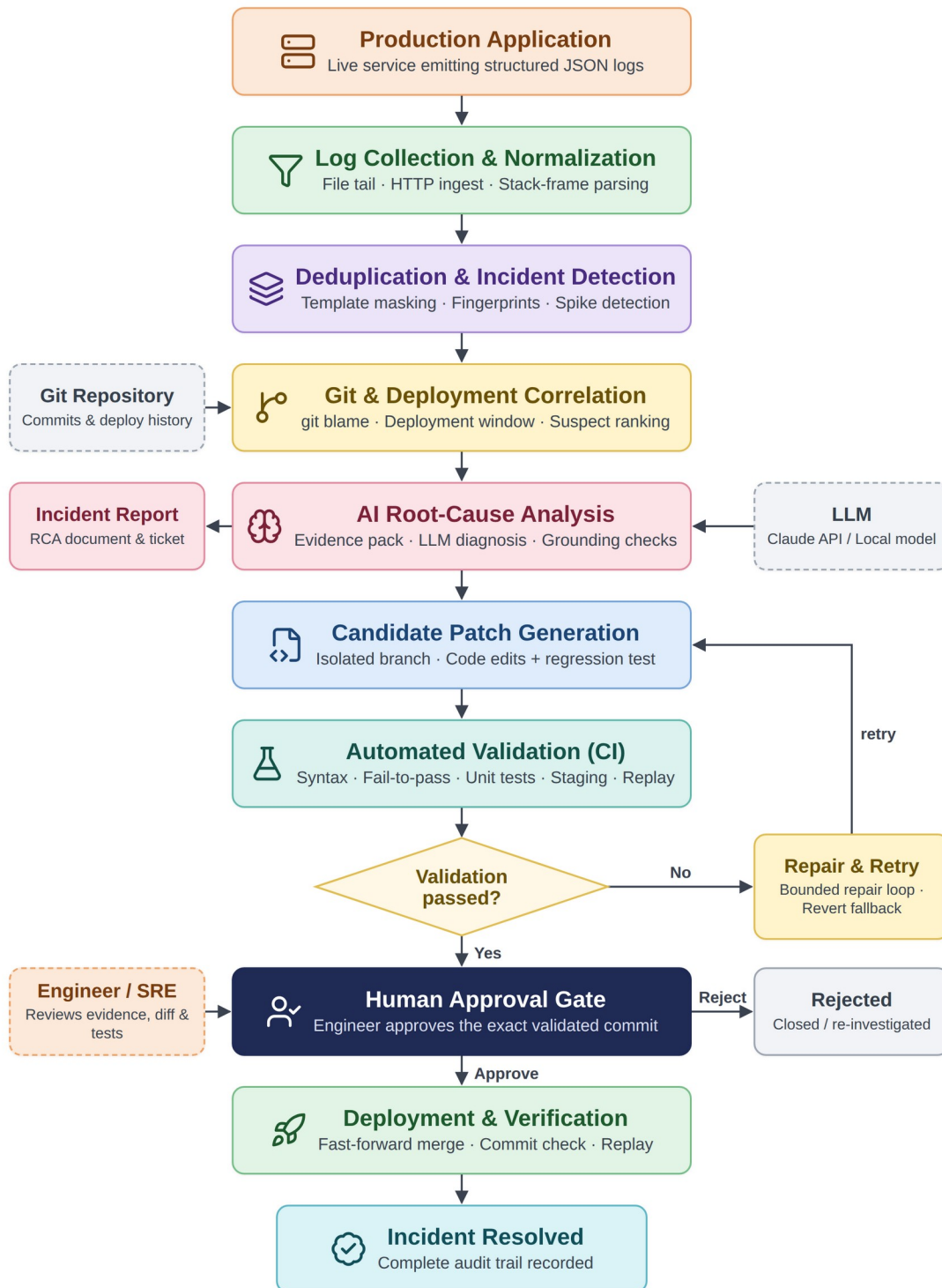
Keywords: AIOps, Observability, Incident Management, Log Analysis, Root-Cause Analysis, Large Language Models, Automated Program Repair, CI/CD, Human-in-the-Loop, DevOps

1. PROBLEM DEFINITION

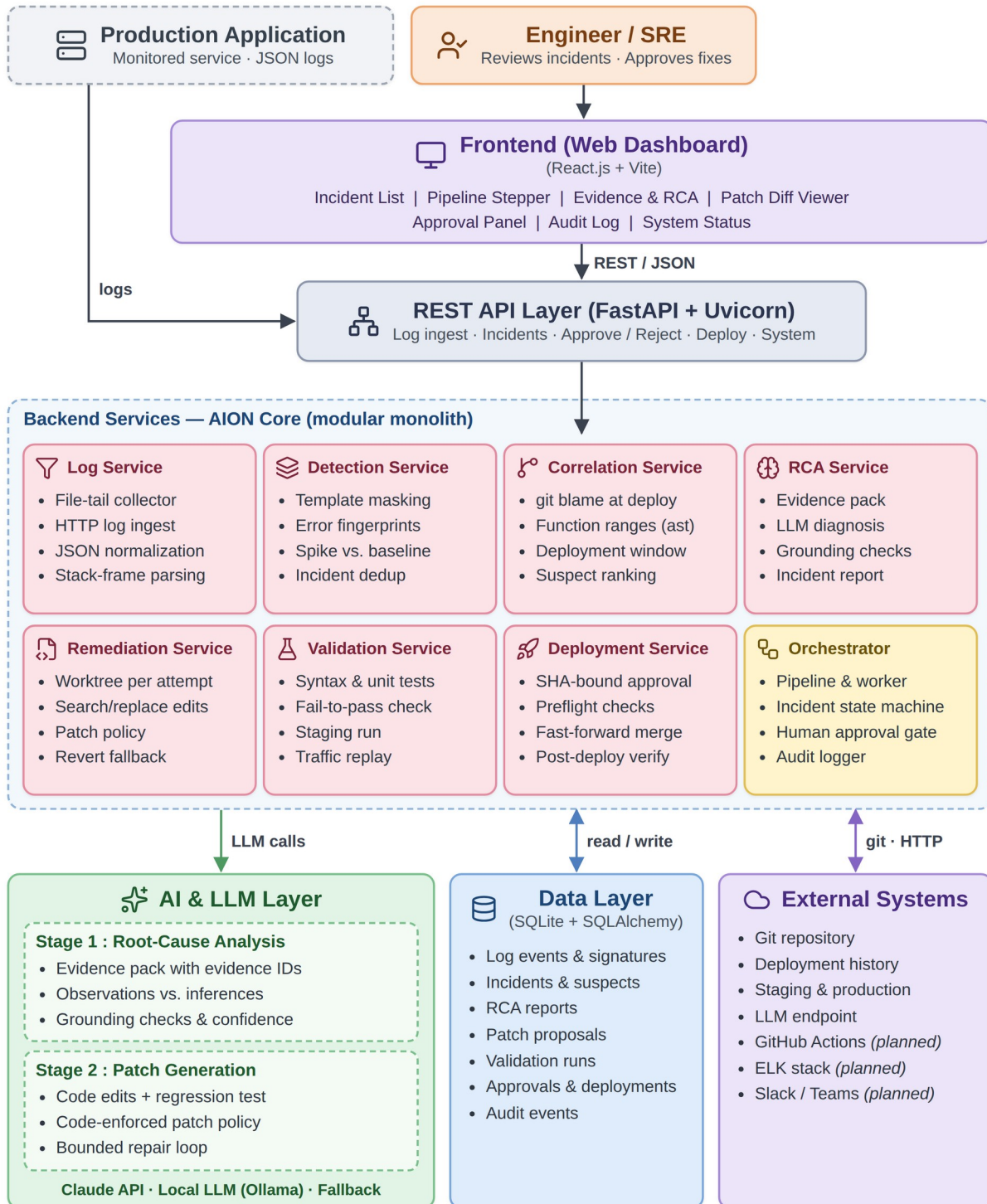
Design and development of an AI-powered incident observation and remediation platform that collects and analyses production application logs, detects and deduplicates incidents, correlates them with Git commits and deployment history, performs evidence-based root-cause analysis using Large Language Models, generates candidate code patches that are validated through automated testing and staging, and deploys a fix to production only after explicit human approval.

2. PROCESS DIAGRAM, SYSTEM ARCHITECTURE, MODULES & FUNCTIONALITIES

a) Process Diagram



b) System Architecture



c) Modules

Module 1: Log Collection and Normalization

Purpose: Collect application logs from production services and convert them into a uniform, structured form for analysis.

Key Features:

- File-tail log collector for running services
- HTTP log-ingest API for external log sources
- Normalization of structured JSON log records
- Extraction of service, level, message, request and timestamp fields
- Parsing of stack traces into individual frames (file, line, function)
- Registration of monitored services and recording of deployment events

Module 2: Log Deduplication and Incident Detection

Purpose: Group repeated errors into unique signatures and decide when an error pattern becomes an incident.

Key Features:

- Template masking of variable values such as IDs, numbers and timestamps
- Error fingerprinting using the exception type and stack frames
- Signature table with occurrence counts and first/last-seen times
- Spike detection: current error rate compared with a historical baseline
- Configurable detection window, minimum error count and spike ratio
- Incident-level deduplication, so repeated errors attach to the open incident
- No re-detection from late log lines after an incident is closed

Module 3: Git and Deployment Correlation

Purpose: Identify the code changes that most probably caused the incident.

Key Features:

- Analysis of recent commits and the deployment history of the service
- git blame of the failing lines at the deployed revision
- Function-level code ranges using Python ast analysis
- Deployment-window, file-overlap and code-dependency signals
- Explainable weighted scoring with a reason for every signal
- Ranked list of suspect commits for the incident

Module 4: AI Root-Cause Analysis

Purpose: Produce an evidence-based explanation of why the incident occurred.

Key Features:

- Curated evidence pack: log clusters, stack trace, deployments, suspect commits, source code and similar past incidents
- Stable evidence IDs that the model must cite
- Provider-neutral LLM layer (Claude API or a local OpenAI-compatible model)
- Structured output that separates observations from inferences
- Grounding checks that reject claims not supported by evidence
- Confidence score for the diagnosis
- Clearly labelled deterministic fallback analyzer when no LLM is configured

Module 5: Incident Report and Ticket Generation

Purpose: Document every incident in a consistent, reviewable format.

Key Features:

- Incident summary: affected service, error signature, impact and timeline
- Root cause with cited evidence and the identified culprit commit
- Recommended fix together with its validation results
- Storage of the exact evidence shown to the AI for later review
- Retrieval of similar past incidents and how they were fixed
- Ticket creation in an issue tracker such as GitHub Issues or Jira

Module 6: Candidate Patch Generation

Purpose: Generate a reviewable code fix for the diagnosed root cause.

Key Features:

- Isolated Git worktree and branch for every patch attempt
- LLM-generated search/replace code edits, shown as a unified diff
- Generated regression test that reproduces the incident
- Code-enforced patch policy: protected test, CI and dependency files, and limits on files and lines changed
- Automatic repair of indentation-only errors in AI edits
- Bounded generate–validate–repair loop
- git revert of the suspect commit as a fallback fix

Module 7: Automated Validation (CI and Staging)

Purpose: Verify that a candidate patch fixes the incident without breaking existing behaviour.

Key Features:

- Syntax check of every changed file
- Fail-to-pass check: the regression test fails on the old code and passes on the fix
- Execution of the service's existing unit-test suite
- Deployment of the patched service to staging, with a health check
- Replay of the failing production requests and of baseline requests against staging
- Step-by-step validation report with logs and timeouts
- Validation commands defined by the operator, never by the AI

Module 8: Human Approval Gate

Purpose: Ensure that no change reaches production without an explicit human decision.

Key Features:

- Incident lifecycle implemented as a finite-state machine
- No automated transition from “awaiting approval” to deployment
- Engineer reviews the evidence, root cause, diff and validation results
- Approval bound to the exact validated commit SHA
- Approve, reject, or withdraw an approval before deployment
- Re-run of the investigation after a failed validation or deployment
- Approver name, decision and comments recorded

Module 9: Production Deployment and Verification

Purpose: Deploy the approved fix safely and confirm that the service has recovered.

Key Features:

- Pre-flight checks of the approval, commit and branch state
- GitOps-style fast-forward merge of the approved commit
- Verification that production is running the approved commit
- Replay of the previously failing requests on production
- Incident marked resolved only after successful verification
- Failed deployments reported back for re-investigation

Module 10: Incident Dashboard and Audit Trail

Purpose: Provide one interface for monitoring incidents and reviewing every step AION takes.

Key Features:

- Incident list with status, service and timestamps
- Incident detail view with a live pipeline stepper
- Log, evidence and root-cause analysis views
- Diff viewer for candidate patches
- Approval panel for approve, reject and deploy actions
- Append-only audit log of every automated and human action
- System status page: services, AI mode and worker health

3. LITERATURE SURVEY

Modern software systems generate very large volumes of logs, metrics and alerts. Research in **AIOps**, the application of artificial intelligence to IT operations, studies how this data can support failure management: failure prevention, detection, root-cause analysis and remediation [1], [12].

Early work focused on understanding logs. Log-parsing techniques such as Drain convert raw log lines into structured templates using a fixed-depth parse tree [2], while deep-learning approaches such as DeepLog [3] and LogAnomaly [4] learn normal log sequences and flag deviations as anomalies. Recent studies apply **Large Language Models to log analysis** [13] and demonstrate LLM-based anomaly detection on real-world system logs [14]. These methods detect that something is wrong, but they generally stop at detection.

A second line of research addresses **root-cause analysis**. MicroRCA localises the root cause of performance issues in microservices [5], and later work progresses from anomaly detection towards automated log labelling and root-cause analysis [6]. LLM-based approaches for cloud incidents [7] and LLM agents for root-cause analysis [10] show that language models can diagnose incidents when given relevant context, but also expose risks such as hallucinated root causes and telemetry volumes that exceed the model’s context window [15].

A third line of research studies **automated program repair**. A systematic review shows the rapid growth of LLM-based repair techniques [9]; RepairAgent demonstrates an autonomous LLM agent that gathers information, proposes fixes and validates them with tests [8]; and a study on vulnerability repair shows that patch-validation feedback improves the quality of LLM-generated fixes [11]. These techniques are, however, usually evaluated on benchmark bugs, disconnected from live production telemetry, deployment history and CI/CD pipelines.

Overall, existing research shows a progression from **log monitoring** → **anomaly detection** → **root-cause analysis** → **LLM-based program repair**, but each stage is usually studied and built separately, and validating and safely deploying a fix remain manual. AION combines the stages into one closed-loop workflow: log detection → Git and deployment correlation → evidence-cited root-cause analysis → candidate patch → CI and staging validation → human-approved deployment. Every AI output is checked deterministically and every production change requires human approval.

SIMILAR SYSTEMS / PRODUCTS AVAILABLE:

I. ELK Stack (Elasticsearch, Logstash, Kibana)

Pros: Collects, indexes, searches and visualises large volumes of application logs, with dashboards and alerting.

Cons: Focused on log storage, search and visualisation; root-cause analysis, code fixing and validation of the fix remain manual work for engineers.

II. Sentry

Pros: Groups application errors with their stack traces, links them to releases and suspect commits, and offers AI-assisted issue analysis and fix suggestions.

Cons: Fixes are proposed as code changes or pull requests; staging validation with request replay, an approval gate bound to the validated commit, and verified production deployment are left to the team's own CI/CD process.

III. PagerDuty

Pros: Provides alerting, on-call scheduling, incident coordination and event grouping that reduces alert noise.

Cons: Concentrates on notifying and coordinating responders; finding the faulty code change and writing, testing and deploying a fix are left to the engineers.

4. **OBJECTIVES**

- To collect and normalize production application logs from file and HTTP sources.
- To group repeated errors into unique signatures using template masking and error fingerprints.
- To detect incidents automatically as error spikes against a historical baseline.
- To correlate each incident with Git commits and deployment history to identify the probable culprit commit.
- To perform evidence-based root-cause analysis using Large Language Models with grounding checks.
- To generate a structured incident report and ticket for every detected incident.
- To generate candidate code patches as reviewable diffs on isolated Git branches.
- To generate regression tests that reproduce the incident and prove the fix.
- To validate every patch through syntax checks, unit tests, staging deployment and request replay.
- To enforce a human approval gate, bound to the exact validated commit, before production deployment.
- To deploy approved fixes and verify the recovery of the production service.
- To maintain a complete, append-only audit trail of all automated and human actions.
- To support both a cloud LLM (Claude API) and local LLMs, so that sensitive logs can stay on premises.
- To evaluate the system on realistic bug scenarios with known root causes.
- To provide an integrated dashboard for monitoring and managing the complete incident lifecycle.

5. SCOPE OF PROJECT

- **Log Ingestion:** Collection and normalization of structured logs from monitored services through file tailing and an HTTP API.
- **Incident Detection:** Deduplication of errors into signatures and detection of error spikes against a baseline.
- **Change Correlation:** Ranking of suspect commits using Git history and deployment records.
- **Root-Cause Analysis:** LLM-based, evidence-cited diagnosis with a deterministic fallback.
- **Incident Documentation:** Incident reports and tickets for every detected incident.
- **Patch Generation:** AI-generated code fixes with regression tests, or a revert of the suspect commit.
- **Automated Validation:** CI-style testing, staging deployment and request replay for each patch.
- **Human Approval:** Approve / reject workflow bound to the validated commit.
- **Deployment & Verification:** GitOps-style deployment of approved fixes and verification on production.
- **Dashboard & Audit:** Web dashboard and append-only audit trail of the full incident lifecycle.
- **Evaluation:** Measurement of detection, root-cause and patch accuracy on seeded bug scenarios.
- **Boundary of the first version:** Python services and stack traces, with local staging and production environments; Kubernetes, GitHub Actions and other languages are planned extensions.

6. SOFTWARE & HARDWARE REQUIREMENTS

HARDWARE REQUIREMENTS:

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent
- **RAM:** Minimum 8 GB; recommended 16 GB (for running a local LLM)
- **Storage:** SSD with at least 20 GB free space for source code, databases and local model files
- **GPU (optional):** NVIDIA GPU with 6 GB or more VRAM for running a local LLM
- **Internet:** Stable connection for GitHub and cloud LLM API calls
- **Display:** Minimum 1920 × 1080 resolution

SOFTWARE REQUIREMENTS:

❖ **FRONTEND:**

- **React.js:** Component-based incident dashboard
- **Vite:** Development server and production build tool
- **JavaScript (JSX):** Frontend logic and interaction
- **HTML5 / CSS3:** Page structure, layout and styling
- **Fetch API:** REST communication with the backend, with periodic polling for live updates

❖ **BACKEND:**

- **Python 3.10+:** Primary backend programming language
- **FastAPI:** REST APIs with request validation and automatic OpenAPI (Swagger) documentation
- **Uvicorn:** ASGI server for the FastAPI application
- **Pydantic:** Request and response data validation
- **SQLAlchemy 2.0:** Object-relational mapping for database access
- **httpx:** HTTP client for LLM providers, health checks and request replay
- **pytest:** Unit testing and automated patch validation

❖ **DATABASE:**

- **SQLite (WAL mode):** Storage of services, log events, signatures, incidents, suspect commits, RCA reports, patches, validation runs, approvals, deployments and audit events
- **PostgreSQL (planned):** Multi-worker, production-scale deployments

❖ **AI & LLM LAYER:**

- **Claude API (Anthropic SDK):** Root-cause analysis and patch generation with structured outputs
- **Local LLM (Ollama / LM Studio):** OpenAI-compatible local models such as qwen2.5-coder for offline and private use

- **Evidence Pack:** Curated, bounded context with evidence IDs instead of raw log dumps
- **Grounding Checks:** Verification that every AI claim cites real evidence
- **Deterministic Fallback:** Rule-based analyzer and git revert when no LLM is configured

❖ **EXTERNAL INTEGRATIONS:**

- **Git:** Commit history, blame, worktrees, branches and reverts
- **Observability Sources:** Structured JSON log files and HTTP ingest; Elasticsearch / Kibana (ELK) integration planned
- **CI/CD:** Local validation runner; GitHub Actions / Jenkins integration planned
- **Deployment:** GitOps-style fast-forward deployment; Kubernetes / ArgoCD planned
- **Notifications:** Slack / Microsoft Teams bot (planned)

❖ **DEVELOPMENT ENVIRONMENT:**

- **Operating System:** Windows / Linux
- **IDE:** VS Code
- **Version Control:** Git and GitHub
- **Runtimes:** Python 3.10+, Node.js 18+ and npm
- **API Testing:** Swagger UI / Postman
- **Documentation:** Markdown documentation and a technical decision log

❖ **SECURITY REQUIREMENTS:**

- **Human Approval:** No production deployment without an approval bound to the validated commit SHA
- **Patch Policy:** Protected test, CI and dependency files; limits on the files and lines a patch may change
- **Isolation:** Separate Git worktree and process, with timeouts, for every patch attempt (container sandbox planned)
- **Command Safety:** Validation commands defined by the operator and executed without a shell
- **Data Protection:** Local LLM option for sensitive logs; secret and PII redaction before prompting (planned)
- **Access Control:** Authentication and role-based approver access (planned)
- **Auditability:** Append-only audit trail of every action

7. EXPECTED OUTCOMES

1) Automatic Incident Detection

Detect production incidents automatically from application logs, grouping repeated errors into a single incident and filtering out harmless noise.

2) Accurate Change Correlation

Rank the commits and deployments most likely to have caused an incident, with an explanation for every score.

3) Evidence-Based Root-Cause Analysis

Provide a clear, evidence-cited explanation of the root cause that an engineer can verify quickly.

4) Reviewable Code Fixes

Generate candidate patches as readable diffs with regression tests, ready for engineer review.

5) Only Validated Changes Reach Approval

Only patches that pass automated tests, staging deployment and request replay are presented for approval.

6) Safe, Human-Approved Deployment

Deploy fixes to production only after explicit human approval of the exact validated commit, and verify the recovery.

7) Reduced Resolution Time and Effort

Reduce the mean time to resolution (MTTR) and the repetitive manual engineering work during incidents.

8) Complete Auditability

Keep a full audit trail of every automated and human action for review and accountability.

8. PROBABLE DATE OF COMPLETION

Academic Year 2026–27

The project is planned for completion within the academic year, covering system design, development, integration, testing, evaluation and final documentation.

9. REFERENCES

- [1] P. Notaro, J. Cardoso, and M. Gerndt, “A Survey of AIOps Methods for Failure Management,” *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, Art. no. 81, 2021. Available: <https://doi.org/10.1145/3483424>
- [2] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in *Proc. IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40. Available: <https://doi.org/10.1109/ICWS.2017.13>
- [3] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298. Available: <https://doi.org/10.1145/3133956.3134015>
- [4] W. Meng et al., “LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs,” in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745. Available: <https://doi.org/10.24963/ijcai.2019/658>
- [5] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, “MicroRCA: Root Cause Localization of Performance Issues in Microservices,” in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [6] T. Wittkopp, A. Acker, and O. Kao, “Progressing from Anomaly Detection to Automated Log Labeling and Pioneering Root Cause Analysis,” in *Proc. AIOps Workshop, IEEE International Conference on Data Mining (ICDM)*, 2023. Available: <https://arxiv.org/abs/2312.14748>
- [7] Y. Chen et al., “Automatic Root Cause Analysis via Large Language Models for Cloud Incidents,” in *Proc. European Conference on Computer Systems (EuroSys)*, 2024.
- [8] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An Autonomous, LLM-Based Agent for Program Repair,” in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. Available: <https://arxiv.org/abs/2403.17134>
- [9] Q. Zhang et al., “A Systematic Literature Review on Large Language Models for Automated Program Repair,” *arXiv:2405.01466*, 2024. Available: <https://arxiv.org/abs/2405.01466>
- [10] D. Roy et al., “Exploring LLM-based Agents for Root Cause Analysis,” in *Proc. ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024.

- [11] U. Kulsum, H. Zhu, B. Xu, and M. d’Amorim, “A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback,” in Proc. 1st ACM International Conference on AI-Powered Software (AIware), 2024. Available: <https://arxiv.org/abs/2405.15690>
- [12] L. Zhang, T. Jia, M. Jia, et al., “A Survey of AIOps in the Era of Large Language Models,” ACM Computing Surveys, 2026. Available: <https://doi.org/10.1145/3746635>
- [13] Z. Ma, J. Yang, and T.-H. Chen, “LLM4Log: A Systematic Review of Large Language Model-based Log Analysis,” arXiv:2604.16359, 2026. Available: <https://arxiv.org/abs/2604.16359>
- [14] M. De la Cruz Cabello, T. P. Sales, and M. R. Machado, “Log Anomaly Detection in AIOps: A Real-World Implementation Using Large Language Models,” Systems and Soft Computing, vol. 8, Art. no. 200475, 2026. Available: <https://doi.org/10.1016/j.sasc.2026.200475>
- [15] “Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation,” in Proc. 3rd International Conference on Artificial Intelligence, Systems and Network Security, 2025. Available: <https://doi.org/10.1145/3797161.3797213>

Name & Sign of Student

Name & Sign of Guide

Name & Sign of HoD